

Core Path Algebra for Graph Queries

A. Nonymous

University of Stuttgart `stxxxxxx@stud.uni-stuttgart.de`

Abstract. Graph databases are well suited for data with many relationships, and path queries are an important part of modern graph query languages. Languages such as GQL and SQL/PGQ provide constructs for finding and manipulating paths, but a common algebraic way of describing these operations is still developing. This paper examines the path algebra proposed by Angles et al. [2]. The algebra treats paths as first-class query objects and provides operators for selection, union, concatenation, and recursion. The paper explains these operators and shows how they can be used to describe path queries and path semantics found in GQL and SQL/PGQ. It also compares the path algebra with relational algebra, automata-based approaches to regular path queries, and semiring-based models. The comparison shows how the path algebra relates to existing formalisms while focusing specifically on the construction and manipulation of paths. Finally, the paper discusses the strengths and limitations of the algebra, particularly the challenges of efficiently evaluating recursive path queries.

Keywords: Graph Databases · Graph Query Languages · Path Algebra · GQL · SQL/PGQ

Table of Contents

Core Path Algebra for Graph Queries	1
<i>A. Nonymous</i>	
1 Introduction.....	3
2 Background	4
2.1 Property Graph Model	4
2.2 Paths in Graphs.....	4
2.3 Graph Query Languages	4
3 Core Path Algebra	5
3.1 Core Operators	5
3.2 Selection	5
3.3 Union.....	6
3.4 Path Concatenation	6
3.5 Closure and Compositionality	6
4 Recursive Path Algebra	6
4.1 Recursive Operator	7
4.2 Path Semantics	7
4.3 Key Issue: Cyclic Graphs and Infinite Expansions	7
5 Mapping to GQL and SQL/PGQ	8
5.1 Selectors and Restrictors	8
5.2 Algebraic Encoding	8
5.3 Example Translation	8
5.4 Expressiveness	8
6 Relation to Formal Foundations.....	9
6.1 Relation to Relational Algebra	9
6.2 Relation to Automata Theory.....	9
6.3 Relation to Semiring Frameworks.....	9
7 Discussion	9
8 Conclusion	10

1 Introduction

Graph databases have become an important technology for managing highly connected data, with applications in social networks, knowledge graphs, recommendation systems, and biological networks [3, 4]. Unlike relational systems, graph databases treat relationships as first-class elements, which makes them suited for queries that require navigation through connected data.

Modern graph query languages such as Cypher, SPARQL, SQL/PGQ, and GQL provide support for path-oriented queries [3]. Such queries allow users to express reachability, pattern matching, and recursive navigation over graph structures. This makes path queries an important part of graph query languages and raises the question of how such operations can be described in a common algebraic framework.

For example, consider finding all paths between two nodes whose sequence of edge labels consists of one or more **KNOWS** relationships followed by zero or more **LIKES** relationships. Such path patterns can be described using regular expressions over edge labels and are common in graph query languages. Classical relational algebra does in contrast not provide a direct operator for recursive path traversal or transitive closure.

To illustrate, consider the following graph:

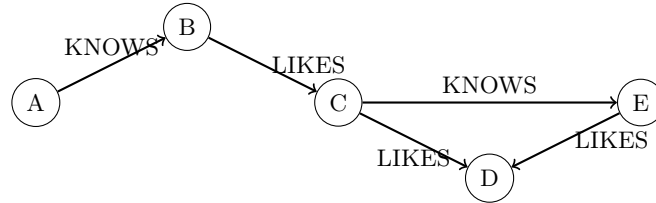


Fig. 1. Example graph illustrating a path query.

In this graph, we seek all paths from node **A** to node **D** whose edge-label sequence matches the regular expression

$$\text{KNOWS}^+ \text{LIKES}^*.$$

$$A \xrightarrow{\text{KNOWS}} B \xrightarrow{\text{LIKES}} C \xrightarrow{\text{LIKES}} D$$

This first path matches the pattern: it contains one **KNOWS** edge followed by two **LIKES** edges.

$$A \xrightarrow{\text{KNOWS}} B \xrightarrow{\text{LIKES}} C \xrightarrow{\text{KNOWS}} E \xrightarrow{\text{LIKES}} D$$

This second path does not match because it contains a **KNOWS** edge after a **LIKES** edge.

In graph query languages, such patterns can be expressed using regular path expressions. For example, in GQL-inspired notation one could write:

MATCH $p = (A) \text{--} : \text{KNOWS}^+ \text{LIKES}^* \text{--} (D)$

The essential idea is that the path must match the regular expression $\text{KNOWS}^+ \text{LIKES}^*$. The example illustrates why path queries require operations for concatenation and recursion that are not part of classical relational algebra. Relational algebra provides a foundation for relational query processing and optimization. For graph queries, however, an algebraic foundation for path-based operations is less established, especially when paths themselves are treated as query results. Angles et al. propose a path algebra in which paths are treated as first-class data objects [2]. It provides operators for path selection, concatenation, union, and recursion, and is intended to provide an algebraic basis for path queries in graph query languages such as GQL and SQL/PGQ. This paper examines the core operators of the proposed path algebra and explains how they can be used to represent path queries. It then considers its relationship to relational algebra, automata-based approaches to regular path queries, and semiring-based models for graph problems.

2 Background

2.1 Property Graph Model

Graph databases are commonly based on the property graph model [1]. A property graph is a directed, labeled multigraph in which both nodes and edges may carry labels and key-value properties.

Formally, a property graph is defined as a tuple $G = (N, E, \rho, \lambda, \sigma)$, where N is the set of nodes, E is the set of edges, ρ defines edge connectivity, λ assigns labels, and σ defines property mappings [1]. A path can be represented visually as shown in Figure 2.

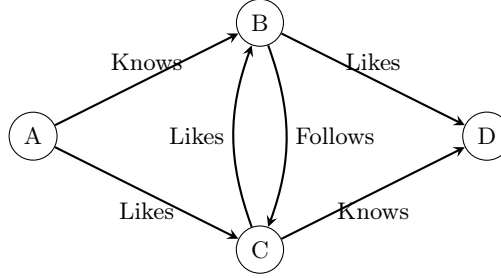


Fig. 2. Simple Property Graph Model Illustration

2.2 Paths in Graphs

A path in a property graph is a sequence of alternating nodes and edges

$$p = (n_1, e_1, n_2, \dots, e_k, n_{k+1})$$

such that each edge connects consecutive nodes. Paths may be classified as walks, trails, acyclic paths, or simple paths depending on whether nodes or edges may repeat, as seen in the table below.

Path queries aim to retrieve sets of such paths satisfying structural or label constraints.

Path Type	Description	Example
Walk	A path in which nodes and edges may be repeated.	$A \rightarrow B \rightarrow C \rightarrow B \rightarrow D$
Trail	A path in which no edge is repeated.	$A \rightarrow B \rightarrow C \rightarrow D$
Acyclic	A path in which no node is repeated.	$A \rightarrow B \rightarrow C \rightarrow D$
Simple	A path in which no node is repeated, except that the first and last nodes may be the same.	$A \rightarrow B \rightarrow C \rightarrow A$

Table 1. Path semantics used in graph query languages [2].

2.3 Graph Query Languages

Modern graph query languages such as Cypher, SPARQL, SQL/PGQ, and GQL support path-oriented queries based on regular expressions over edge labels [3, 2].

In contrast to classical RPQs, these languages aim to return not only reachable node pairs but also the actual witness paths. To control the potentially infinite number of such paths in cyclic graphs, GQL and SQL/PGQ introduce selectors and restrictors, which determine how paths are filtered and how recursion is interpreted [2].

Example Queries:

- **Cypher and GQL** – Matches a KNOWS relationship and returns the path.

```

MATCH p = (x) -[:KNOWS]->(y)
RETURN p

```

- **SQL/PGQ** – Matches a KNOWS relationship and returns the path.

```

SELECT p
FROM GRAPH_TABLE (
  my_graph
  MATCH p = (x) -[:KNOWS]->(y)
  COLUMNS (p)
)

```

3 Core Path Algebra

The core idea of the path algebra is to treat paths as first-class data objects, similar to tuples in relational algebra [2]. The operators take sets of paths as input and return sets of paths, which allows their results to be combined into larger expressions.

3.1 Core Operators

The core algebra consists of three fundamental operators [2]:

- **Selection** (σ): filters paths based on predicates over node or edge labels and properties.
- **Union** ($\cup$): combines two sets of paths.
- **Concatenation (Join)** ($\bowtie$): concatenates paths whose endpoints are compatible.

To illustrate these operators, we use the same running example throughout this section.

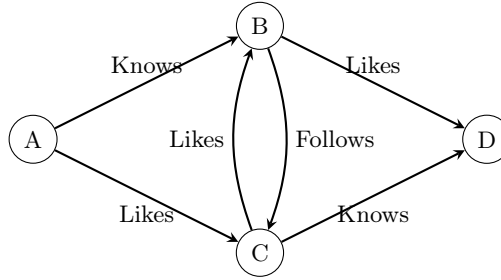


Fig. 3. Example graph used throughout this section.

Assume the set S contains all paths of length two starting from node A :

$$p_1 : A \rightarrow B \rightarrow D \quad p_2 : A \rightarrow C \rightarrow D \quad p_3 : A \rightarrow B \rightarrow C \quad p_4 : A \rightarrow C \rightarrow B$$

3.2 Selection

Selection filters a set of paths according to predicates over nodes, edges, or their properties and is defined as follows:

$$\sigma_c(S) = \{p \in S \mid c(p)\}$$

As an example, we filter the path set according to the label of the first edge.

$$\sigma_{\text{Knows}}(S) = \{p_1, p_3\}$$

$$\sigma_{\text{Likes}}(S) = \{p_2, p_4\}$$

The result is two subsets of S , each containing paths that satisfy the corresponding predicate. Unlike graph traversal itself, selection does not modify paths; it only filters the existing path set.

3.3 Union

Union combines two path sets into a single result.

Using the subsets obtained above,

$$S_1 = \sigma_{\text{Knows}}(S), \quad S_2 = \sigma_{\text{Likes}}(S),$$

their union reconstructs the original path set:

$$S_1 \cup S_2 = \{p_1, p_2, p_3, p_4\} = S.$$

This illustrates that path sets from different query operations can be combined using the usual set-union semantics.

3.4 Path Concatenation

Concatenation joins two compatible path fragments into a longer path. Two paths are compatible if the last node of the first path equals the first node of the second path.

Using the running example, we first consider shorter path fragments:

$$A \rightarrow B, \quad A \rightarrow C,$$

and

$$B \rightarrow D, \quad C \rightarrow D.$$

Concatenating compatible fragments produces longer paths:

$$(A \rightarrow B) \circ (B \rightarrow D) = A \rightarrow B \rightarrow D = p_1,$$

$$(A \rightarrow C) \circ (C \rightarrow D) = A \rightarrow C \rightarrow D = p_2.$$

A query can therefore construct longer paths from smaller path fragments. This is similar to a relational join, but the result is a path rather than a tuple.

3.5 Closure and Compositionality

A central property of the core algebra is closure: every operator returns another set of paths. Consequently, the output of one operator can immediately serve as the input to another.

In the running example, the selection operator produced the path sets

$$S_{\text{Knows}} \quad \text{and} \quad S_{\text{Likes}},$$

which could subsequently be combined again using union or further extended using concatenation. Since every intermediate result remains a valid path set, complex path queries can be built from simpler expressions in a modular way.

This compositionality is one of the main strengths of the algebra. It allows query evaluation to be decomposed into reusable intermediate results, supports algebraic query optimization, and provides a clean logical foundation for graph query processing [2].

4 Recursive Path Algebra

A central requirement of graph query languages is the ability to express recursive navigation over paths of unbounded length. In the path algebra, recursion is represented by the operator ϕ , which repeatedly applies path concatenation to construct paths of increasing length [2]. The recursive operator can be understood as a repeated self-join: starting with a set of paths, it combines the paths with the original set and continues until no new paths are produced. This corresponds to the Kleene plus operation used in regular expressions [2]. Intuitively, recursion repeatedly extends short paths into longer ones by concatenating compatible path fragments. Starting from a base set of edges, each iteration adds exactly one additional step, gradually generating longer and more complex paths.

4.1 Recursive Operator

Let S be a set of paths. The recursive operator ϕ repeatedly joins the paths in S with the paths generated in the previous iteration [2]:

$$\phi_0(S) = S$$

and for $i > 0$,

$$\phi_i(S) = (\phi_{i-1}(S) \bowtie S) \cup \phi_{i-1}(S).$$

The recursion continues as long as new paths are produced. In other words, the operator starts with the paths in S , repeatedly extends them by concatenating paths from S , and stops when an iteration does not add any new paths. The resulting operator corresponds to the Kleene plus operation, producing paths of one or more repetitions of the input pattern [2].

To illustrate the recursive process, consider the following base set of edges:

$$S = \{A \rightarrow B, B \rightarrow C, C \rightarrow B\}.$$

The first iteration contains the original edges:

$$S^{(1)} = \{AB, BC, CB\}.$$

In the next iteration, compatible paths can be concatenated:

$$S^{(2)} = S^{(1)} \bowtie S = \{ABC, BCB, CBC\}.$$

A further iteration produces longer paths, for example:

$$S^{(3)} = S^{(2)} \bowtie S = \{ABCB, BCBC, CBCB\}.$$

Thus, repeated application of the join operator generates progressively longer paths.

4.2 Path Semantics

Graph query languages distinguish between different restrictions on path repetition in order to control expressiveness and computational complexity. These semantics can be modeled as constraint operators applied to the recursive closure:

- **Walk semantics:** no restrictions on node or edge repetition; ϕ operates over all finite concatenations.
- **Trail semantics:** edges may not repeat within a path, restricting concatenation to edge-simple expansions.
- **Acyclic semantics:** nodes may not repeat, enforcing cycle-free path construction.
- **Simple path semantics:** no repeated nodes, except that the first and last node may be the same.
- **Shortest path semantics:** selects minimal-length paths from the recursively generated set.

Using the example above, the paths in $S^{(3)}$ and $S^{(4)}$ clearly illustrate that recursion over concatenation naturally produces walks. However, at some point some of these walks are no longer trails or simple paths, since nodes such as B and C are revisited multiple times (e.g., $A \rightarrow B \rightarrow C \rightarrow B$). This highlights why additional semantic restrictions are necessary in practical query languages to prevent uncontrolled path growth.

4.3 Key Issue: Cyclic Graphs and Infinite Expansions

The main difficulty with recursive path evaluation appears in graphs containing cycles. In such graphs, unrestricted recursion can repeatedly traverse the same cycle and therefore produce infinitely many distinct paths. Although the graph itself is finite, the number of possible walks can be infinite [2].

For example, if a path can reach a cycle between two nodes, the recursive operator can continue traversing that cycle and generate paths of increasing length. Under the default walk semantics, the evaluation may therefore never terminate [2].

The path semantics introduced above provide a way to control this behavior. Restricting recursion to trails, acyclic paths, simple paths, or shortest paths limits which paths can be generated and can therefore produce finite results on finite graphs [2].

5 Mapping to GQL and SQL/PGQ

The path algebra can be used to describe path queries from GQL and SQL/PGQ using a common set of operators [2]. In particular, the path modes of these languages can be represented using recursive operators together with grouping, ordering, and projection. This makes it possible to describe the evaluation of a path query as a sequence of algebraic operations.

5.1 Selectors and Restrictors

GQL uses selectors and restrictors to control the paths produced by a path query [2]:

- **Restrictors**: determine which paths are considered during recursive evaluation, such as WALK, TRAIL, SIMPLE, and ACYCLIC.
- **Selectors**: determine which of the resulting paths are returned, such as ALL, ANY, and ANY SHORTEST.

5.2 Algebraic Encoding

Each selector-restrictor combination can be expressed using a combination of:

- recursion operator (ϕ),
- grouping operator (γ),
- ordering operator (τ),
- projection operator (π).

5.3 Example Translation

Consider the following GQL query:

```
MATCH ANY SHORTEST TRAIL
p = (x) - [:KNOWS] ->+(y)
```

The query asks for one shortest trail between each pair of connected nodes. The TRAIL restrictor means that edges cannot be repeated, while ANY SHORTEST selects one shortest path for each source-target pair [2].

The corresponding algebraic evaluation can be understood as a sequence of operations. First, the edges labeled KNOWS are selected. The recursive operator ϕ_{Trail} then generates paths that satisfy the TRAIL semantics. The resulting paths are grouped by their source and target nodes, ordered by path length, and finally projected so that one path is returned from each group.

In simplified form, the structure of the translation is:

$$\pi\left(\tau\left(\gamma\left(\phi_{\text{Trail}}(\sigma_{\text{KNOWS}}(\text{Edges}(G)))\right)\right)\right).$$

The exact parameters of γ , τ , and π determine how paths are grouped, ordered, and selected. This example shows how a GQL path mode can be represented as a composition of algebraic operators [2].

5.4 Expressiveness

The path algebra can represent the selector-restrictor combinations defined by GQL and SQL/PGQ [2]. At the same time, the algebra allows additional combinations because grouping, ordering, projection, and recursion are represented as separate operators.

For example, the algebra can combine a recursive trail operator with grouping by path length and then select one path from each group. Such a combination is not directly provided by the GQL path mode syntax [2].

The authors show that the different choices of grouping, ordering, projection, and recursion result in a larger space of possible expressions than the selector-restrictor combinations defined by GQL. This means that the algebra is not limited to reproducing existing language features, but can also describe path operations that are not directly available in the current standards [2].

6 Relation to Formal Foundations

The path algebra is related to several established formalisms in database and automata theory. Its operators resemble those of relational algebra, while its recursive operator is closely related to the regular expressions used in regular path queries. Semiring-based approaches provide another point of comparison, since they also use algebraic operations to combine paths.

6.1 Relation to Relational Algebra

The path algebra is closely related to relational algebra. Both use operators such as selection (σ), union ($\cup$), and join ($\bowtie$), but they operate on different kinds of objects. In relational algebra, the operators are applied to relations containing tuples, whereas the path algebra operates on sets of paths [5, 2]. In the path algebra, selection filters paths according to predicates over their nodes and edges, union combines path sets, and join concatenates compatible paths. The join operation therefore has a different meaning from a traditional relational join, since its result is a longer path rather than a tuple. The main difference is that paths preserve the order and connectivity of the nodes and edges they contain. This makes the path algebra suitable for navigation-oriented queries, where the path itself can be part of the query result.

6.2 Relation to Automata Theory

Regular path queries (RPQs) are commonly described using regular expressions over edge labels. These expressions can be translated into finite automata, which can then be used to evaluate whether paths in a graph satisfy the given pattern [7].

The path algebra describes similar recursive path patterns using algebraic operators. In particular, the recursive operator ϕ corresponds to repeated path concatenation and is related to the Kleene plus operation of regular expressions. Zero-length paths can additionally be included when Kleene star semantics are required [2].

The two approaches therefore emphasize different aspects of path querying. Automata provide a model for recognizing paths that satisfy a regular expression, while the path algebra describes how sets of paths can be constructed and manipulated using algebraic operators. The approaches are therefore complementary rather than competing formalisms.

6.3 Relation to Semiring Frameworks

Semiring-based approaches provide another algebraic way to model path problems. A semiring defines two operations that can be used to combine alternative paths and to concatenate path weights. Depending on the chosen semiring, this can support tasks such as path counting, shortest-path computation, or probabilistic analysis [8, 6].

The path algebra shares the idea of using algebraic operations for path composition, but its purpose is different. Instead of assigning a value to each path and aggregating these values, the path algebra treats the paths themselves as query objects. Its operators therefore describe how path sets are constructed, filtered, and combined [2].

The two approaches can therefore be viewed as complementary. Semiring frameworks are useful when paths need to be evaluated quantitatively, while the path algebra focuses on the structural and logical representation of path queries.

7 Discussion

The path algebra proposed by Angles et al. provides an algebraic framework for describing path queries in graph query languages [2]. Its main idea is to treat paths as query objects and to define operations for filtering, combining, and recursively extending them. A key advantage of the algebra is its treatment of paths as first-class objects. Since the operators return sets of paths, the result of one operation can be used as the input to another. This makes it possible to construct complex path queries from smaller algebraic expressions [2]. Another important feature is the explicit modeling of recursion through the operator ϕ . Different variants of the recursive operator can represent walk, trail, acyclic, simple, and shortest-path semantics [2].

From a systems perspective, the algebra can serve as a basis for logical query planning. An algebraic expression can be represented as a tree of operations, which makes the individual steps of query evaluation

explicit [2]. This also creates opportunities for query optimization, for example by changing the order of operations or by applying equivalent algebraic expressions. The actual execution of these plans, however, requires physical operators and cost models that are outside the scope of the proposed algebra.

A limitation of the framework is that it does not define physical execution strategies or cost models. In particular, recursive path evaluation can become expensive on large or highly cyclic graphs. Efficient implementations therefore require physical operators and optimization techniques that are not part of the logical algebra itself.

Despite these limitations, the framework represents an important step toward a standardized and implementation-independent foundation for graph query processing.

8 Conclusion

This seminar paper presented the path algebra for graph query languages proposed by Angles et al. [2]. The algebra treats paths as first-class query objects and provides operators for selection, union, concatenation, and recursion. These operators make it possible to describe path queries in a compositional way.

The paper also examined how the algebra relates to path querying features in GQL and SQL/PGQ, including different path selection and restriction semantics. Furthermore, it was compared with relational algebra, automata-based approaches to regular path queries, and semiring-based models. These comparisons show that the path algebra combines ideas from existing formalisms while focusing specifically on the construction and manipulation of paths.

The main strength of the approach is that it provides a common logical framework for describing different types of path queries. At the same time, the algebra does not specify physical execution strategies or cost models. Efficient evaluation of recursive path queries, especially on large and cyclic graphs, therefore remains an important practical challenge.

Overall, the path algebra provides a useful foundation for reasoning about graph path queries and offers a clear connection between the semantics of modern graph query languages and their underlying algebraic operations.

References

1. Angles, R.: The property graph database model. In: Alberto Mendelzon International Workshop on Foundations of Data Management (AMW) (2018)
2. Angles, R., Arenas, M., Barceló, P., et al.: Path-based algebraic foundations of graph query languages. arXiv preprint arXiv:2407.04823 (2024)
3. Angles, R., Arenas, M., Barceló, P., Hogan, A., Reutter, J.L., Vrgoč, D.: Foundations of modern query languages for graph databases. *ACM Computing Surveys* **50**(5), 68:1–68:40 (2017)
4. Bonifati, A., Fletcher, G.H.L., Voigt, H., Yakovets, N.: *Querying Graphs*. Morgan & Claypool (2018)
5. Codd, E.F.: A relational model of data for large shared data banks. *Communications of the ACM* **13**(6), 377–387 (1970)
6. García, R., Angles, R.: An algebra for path manipulation in graph databases. In: *Advances in Databases and Information Systems*. pp. 61–74 (2022)
7. Mendelzon, A.O., Wood, P.T.: Finding regular simple paths in graph databases. *SIAM Journal on Computing* **24**(6), 1235–1258 (1995)
8. Rodriguez, M.A., Neubauer, P.: A path algebra for multi-relational graphs. In: *IEEE 27th International Conference on Data Engineering Workshops*. pp. 128–131 (2011)